



Spring boot

framework para desenvolvimento de aplicações em java e para criação de API Rest

Facilitador

Agiliza o processo de criação de aplicativos, Padrão RESTful

permitindo a
concentração na parte lógica
Prático

Reduz a quantidade de configurações
necessárias

Define como os sistemas
distribuídos podem se comunicar de forma
eficiente pela internet. Facilita a criação de
APIs.

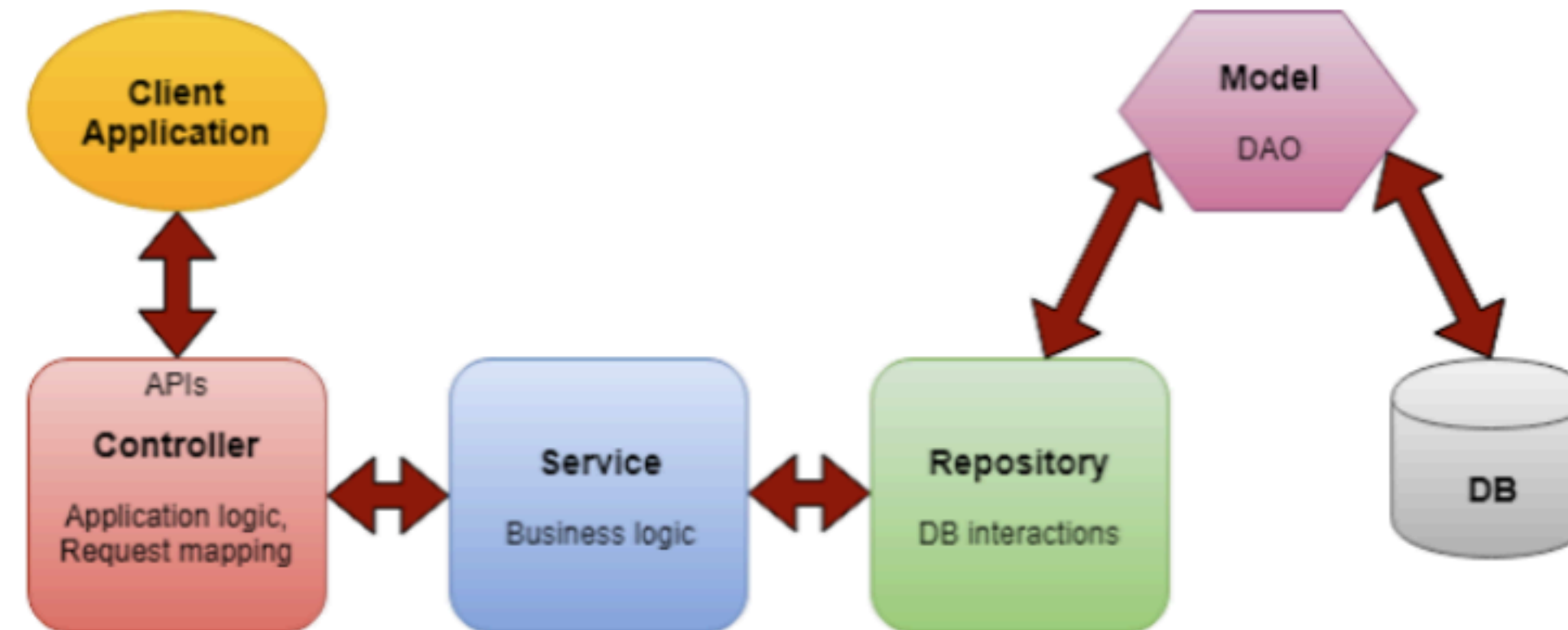
Arquitetura em camadas

Controller

Camada que serve como
intermediário entre Model e a
View, gerenciando as interações
entre os dois componentes.

Model

Define as entidades que



representam os dados da aplicação
Service

realizando operações sobre os dados do
Model.
Repository

Define os métodos de acesso aos dados da
aplicação, utilizando a tecnologia JPA para
interagir com o banco de dados.

Encapsula a lógica de negócio da aplicação,

Controller

@PostMapping e @DeleteMapping são utilizadas para
mapear requisições HTTP para métodos específicos
do Controller.

No Spring Boot, os Controllers são implementados
como classes Java anotadas com
@RestController.

As anotações @GetMapping, @PostMapping,

Model Service

```

@Entity
public class Tarefa {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String titulo;

    private String descricao;

    @Temporal(TemporalType.DATE)
    private Date dataCriacao;

    @Enumerated(EnumType.STRING)
    private StatusTarefa status;

    // Getters, setters e construtores omitidos por brevidade
}

```

```

@Service
public class TarefaService {

    @Autowired
    private TarefaRepository tarefaRepository;

    public List<Tarefa> listarTodasTarefas() {
        return tarefaRepository.findAll();
    }

    public Tarefa buscarTarefaPorId(Long id) {
        return tarefaRepository.findById(id).orElseThrow(() -> new TarefaNotFoundException(id));
    }

    public Tarefa salvarTarefa(Tarefa tarefa) {
        return tarefaRepository.save(tarefa);
    }

    public Tarefa editarTarefa(Tarefa tarefa) {
        return tarefaRepository.save(tarefa);
    }

    public void excluirTarefa(Long id) {
        tarefaRepository.deleteById(id);
    }
}

```

Repository

```

@Repository
public interface TarefaRepository extends JpaRepository<Tarefa, Long> {

}

```

Métodos HTTP

Definida a ação a ser realizada em uma

requisição. Ele é baseado na comunicação entre clientes (com um

navegadorouumaplicativo)eservidores

Usado para buscar dados nos servidor
Não altera nada, apenas consulta

Usado para atualizar um recurso inteiro

GET /usuarios

PUT /usuarios/10
Body: { "nome": "João Silva", "email": "joao@email.com" }

Usado para atualizar parcialmente um recurso

PATCH /usuarios/10

Body: { "nome": "João Souza" }

Usado para enviar dados ao servidor e criar um novo recurso

Usado para remover um recurso do servidor

DELETE /usuarios/10

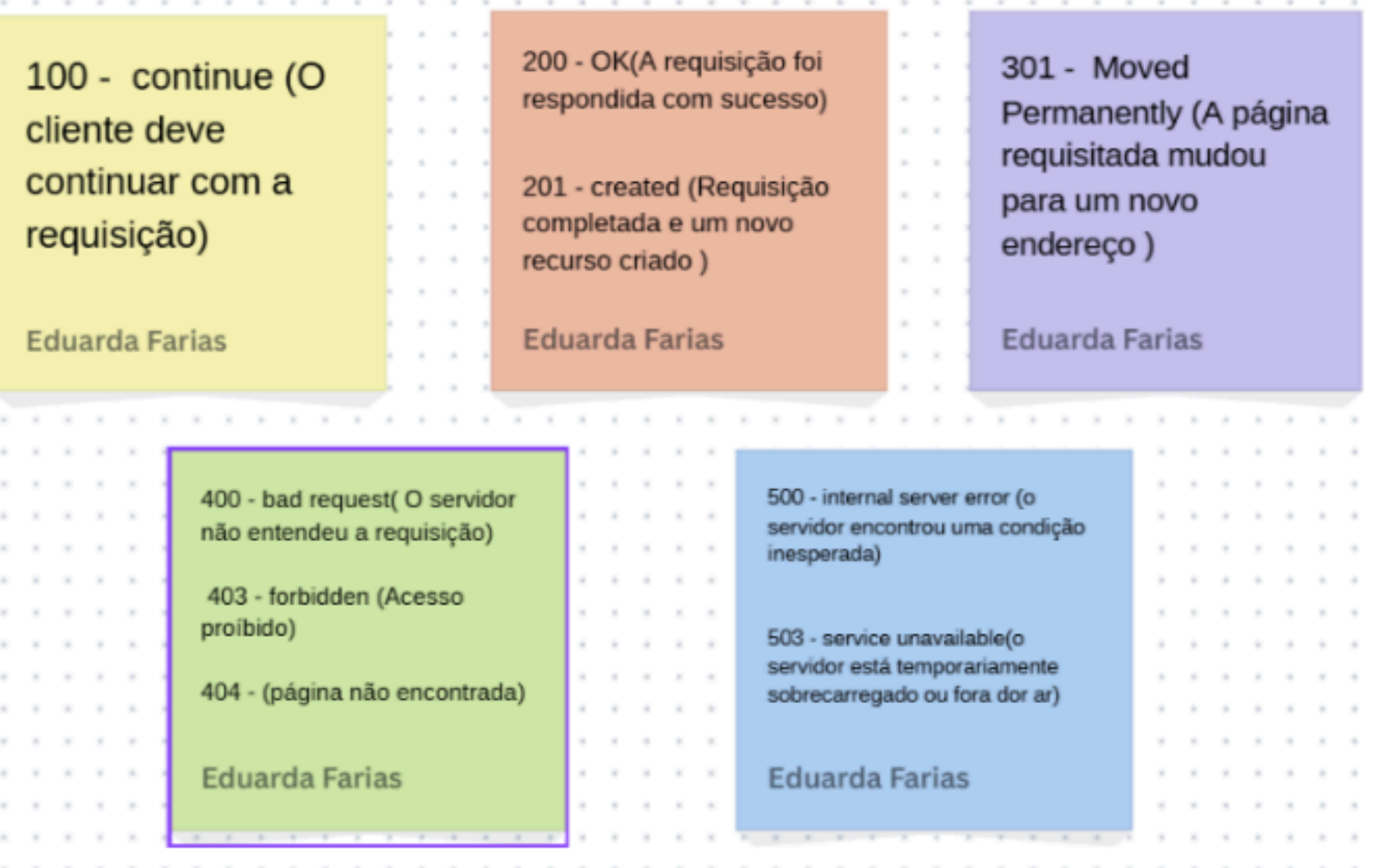
POST /usuarios
Body: { "nome": "João", "email": "joao@email.com" }

Respostas HTTP

As respostas de requisições HTTP são associadas a um código, divididos em 5

classes, cada uma com um significado diferente

<https://http.cat/>



DTOs

Evita expor a entidade do banco

Protege a estrutura interna do sistema

Facilita a conversão de dados

Permite transformar dados antes de enviá-los

Reduz o tráfego de dados Envia

apenas os campos necessários

Melhora a segurança

Esconde informações sensíveis

```
@Id
@GeneratedValue(strategy = GenerationType.
IDENTITY)
private Long id;
private String nome;
private String email;
private String senha;
para o front
}
```

```
public class UsuarioDTO {
    private String nome;
    private String email;

    public UsuarioDTO(Usuario usuario) {
        this.nome = usuario.getNome();
        this.email = usuario.getEmail();
    }

    // Getters
}
```

```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    @Autowired
    private UsuarioRepository usuarioRepository;

    @GetMapping
    public List<UsuarioDTO> listarUsuarios() {
        List<Usuario> usuarios = usuarioRepository.findAll();
        return usuarios.stream().map(UsuarioDTO::new).collect(Collectors.toList());
    }
}
```

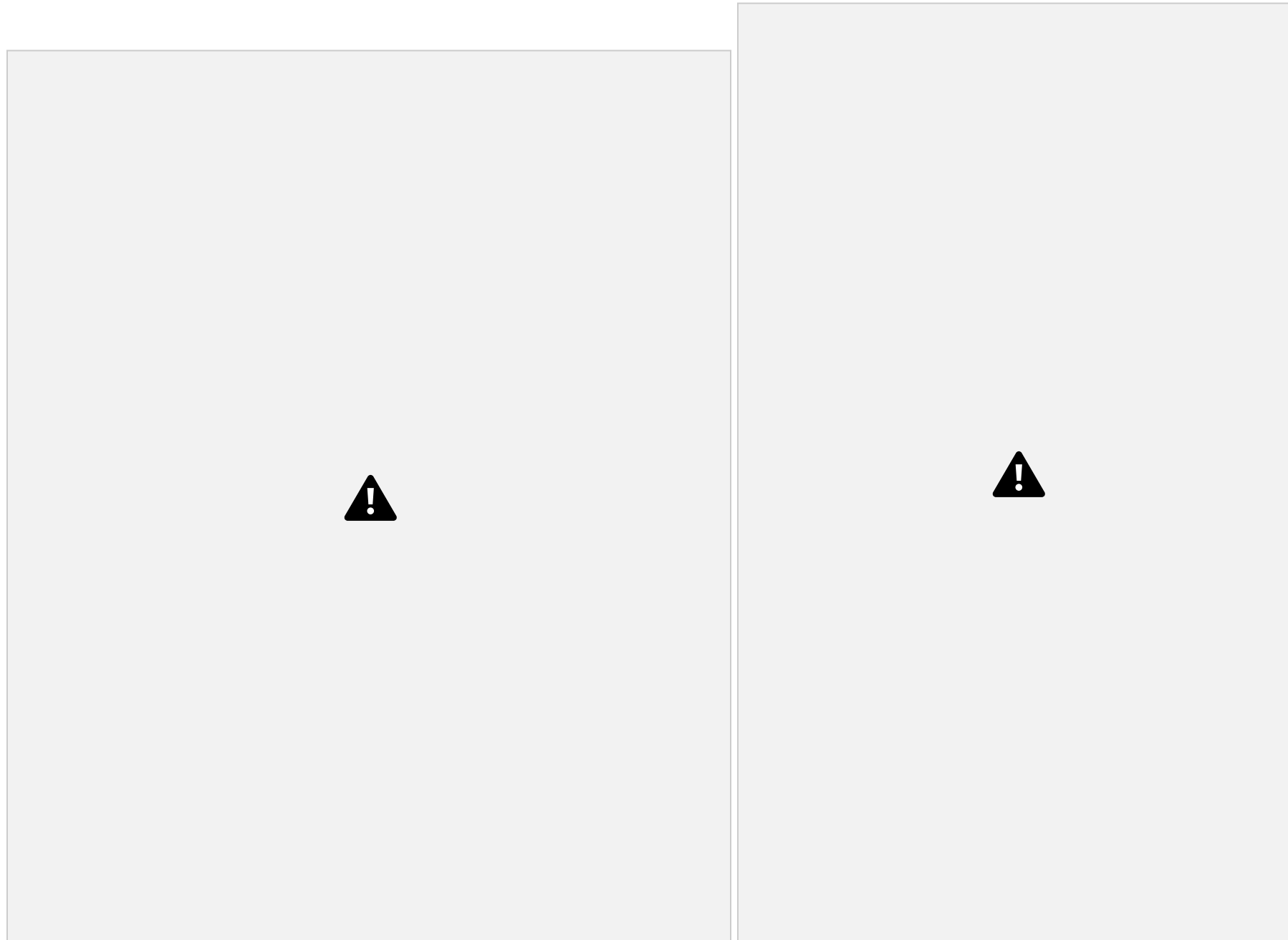
Lombok

@Data

Gera automaticamente getters e setters para todos os atributos da classe, além disso, implementa o equals(), hashCode() e toString


```
Filme.java x
1  package com.filmes.filme.model;
2
3  import lombok.Data;
4  import lombok.Builder;
5  import lombok.AllArgsConstructor;
6  import lombok.NoArgsConstructor;
7
8  @Data
9  @Builder
10 @AllArgsConstructor
11 @NoArgsConstructor
12 public class Filme {
13     private Long id;
14     private String nome;
15     private String genero;
16     private Double avaliacao;
17     private Double duracao;
18
19     public static void main(String[] args) {
20         Filme filme = Filme.builder()
21             .id(1L)
22             .nome("O Poderoso Chefão")
23             .genero("Drama")
24             .avaliacao(9.2)
25             .duracao(175.0)
26             .build();
27
28         System.out.println(filme.toString());
29     }
30 }
```





SpringDataJPA

PartedoframeworkSpringqueajudaa
simplificarousodoJPA(JavaPersistence

API)paratrabalharcombancosdedados
relacionaisindependentedoSGBDusado.
Visareduziraquantidadedecódigo
boilerplate(códigorepetitivoe
desnecessário)queéprecisoescreverpara
lidarcomoperaçõesdeBD.

Arquitetura



JPA (Java Persistence API)

Especificação que define como o Java deve interagir com bancos de dados relacionais. Ele permite mapear classes Java para tabelas no banco de dados e fornece um conjunto de funcionalidades para realizar operações CRUD (Create, Read, Update, Delete).

Hibernate

É uma implementação de ORM - Object-Relational Mapping que faz parte da especificação do JPA. Ele facilita o desenvolvimento de aplicações que interagem com bancos de dados relacionais, eliminando a necessidade de escrever muito código SQL e permitindo que desenvolvedores trabalhem com os dados de forma mais natural, utilizando objetos Java.

JDBC

Uma API padrão do Java que permite que aplicativos Java interajam com bancos de dados relacionais. Ele fornece um conjunto de classes e interfaces para conectar-se a um banco de dados, executar consultas e comandos SQL, e processar os resultados.

Anotações JPA

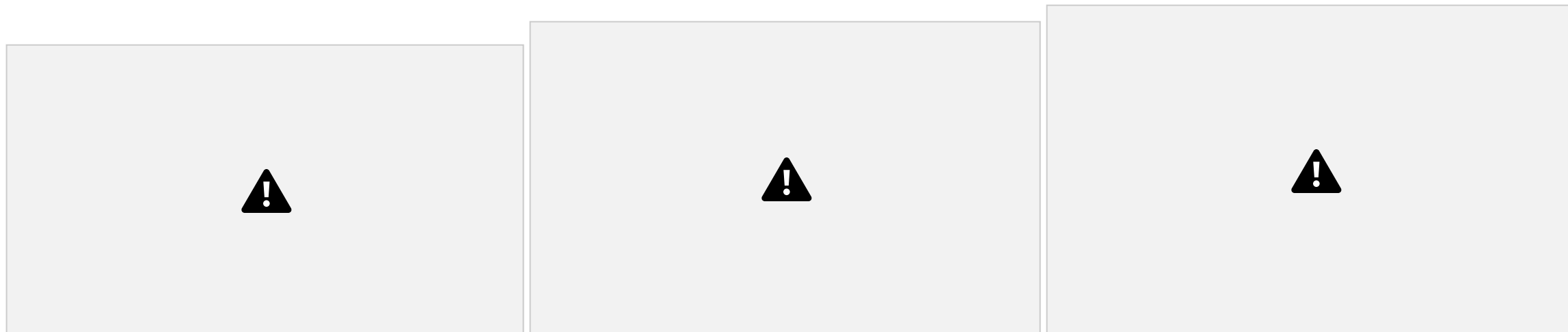
Marca uma classe como uma entidade

Será mapeada para uma tabela no banco de dados

Permite configuração de detalhes, como o nome da tabela.

É aplicada em cima de uma classe que for anotada com `@Entity`

Define o campo de chave primária da entidade



Especifica como o valor da chave

primária é gerado.
Pode ser AUTO, IDENTITY,
SEQUENCE, etc

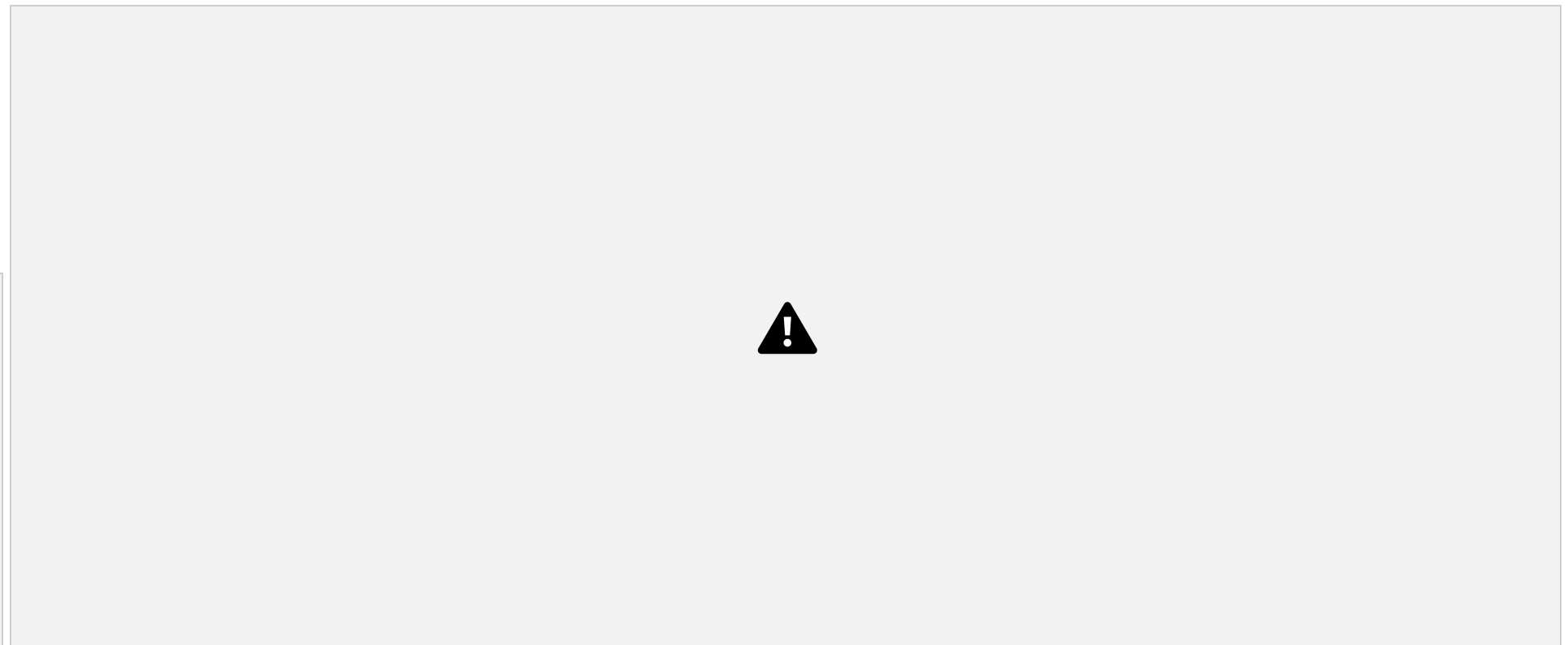
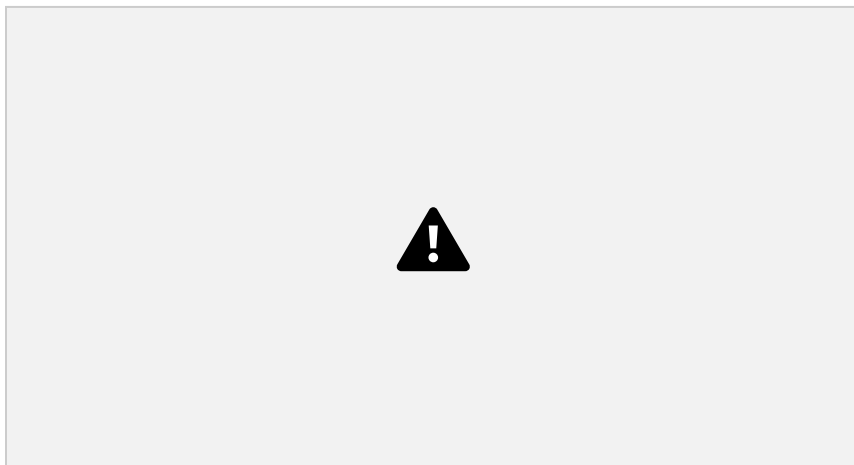
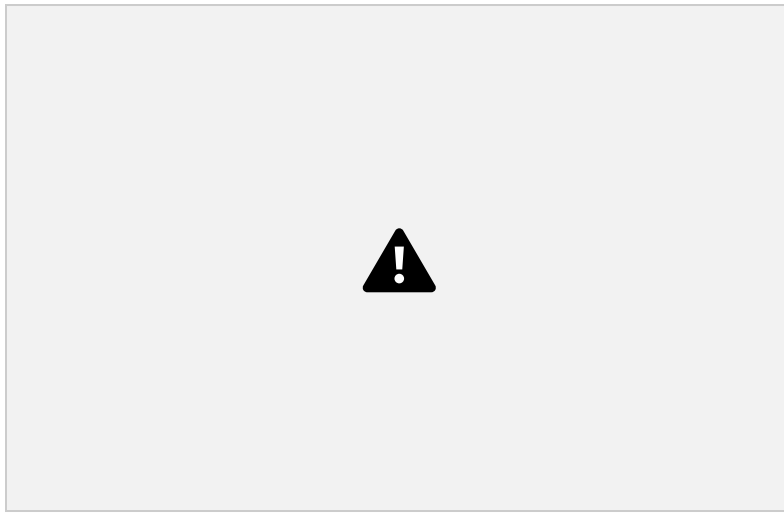
mapeamento de uma coluna. Pode
definir o nome da coluna, se é única,
se pode ser nula, etc.

Especifica os detalhes de



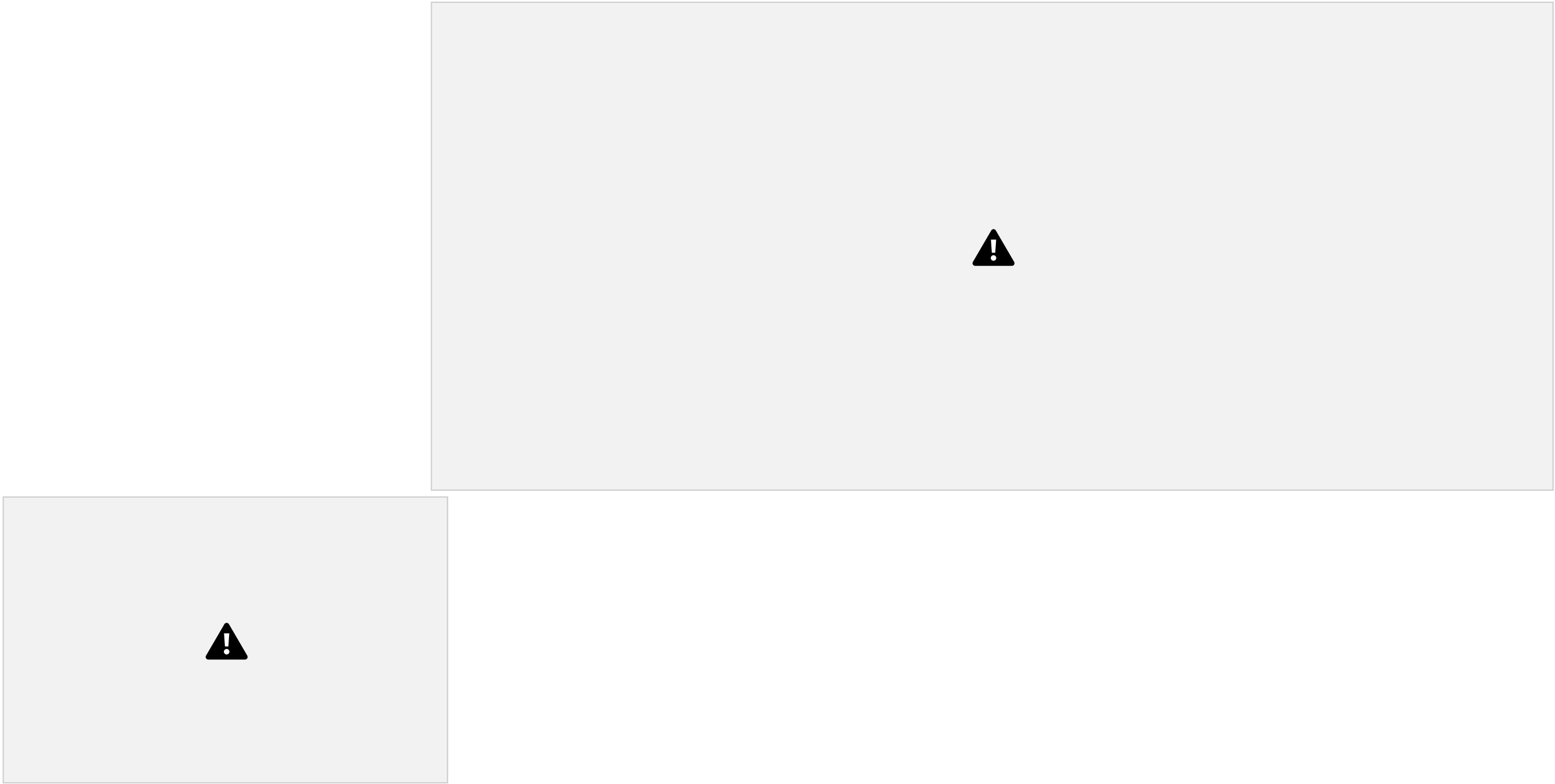
Define um relacionamento um-para
um entre duas entidades





Define um relacionamento um-para
muitos entre duas entidades

Define um relacionamento muitos
para-um entre duas entidades

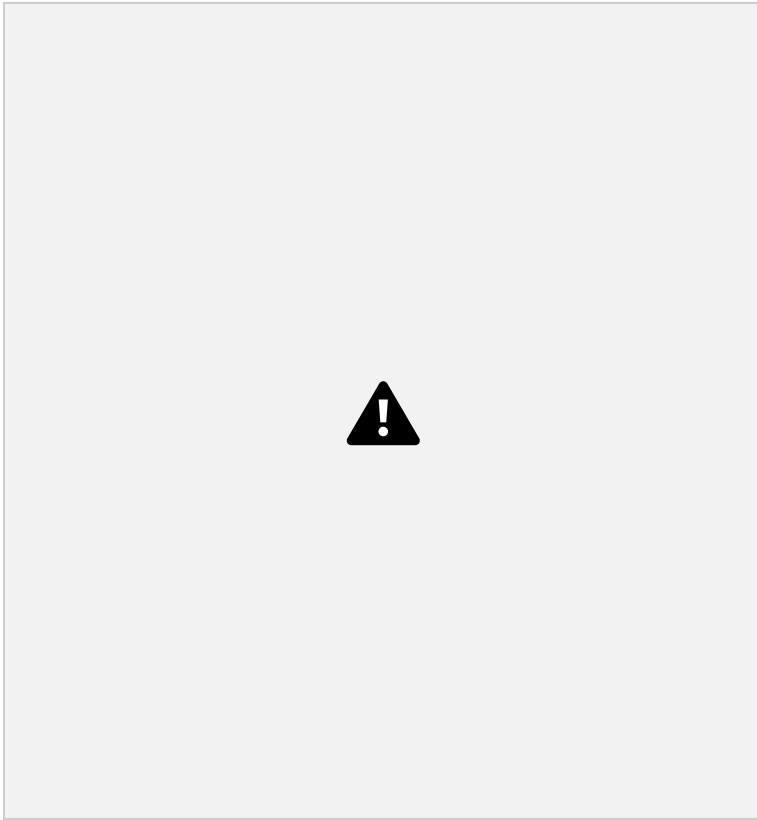
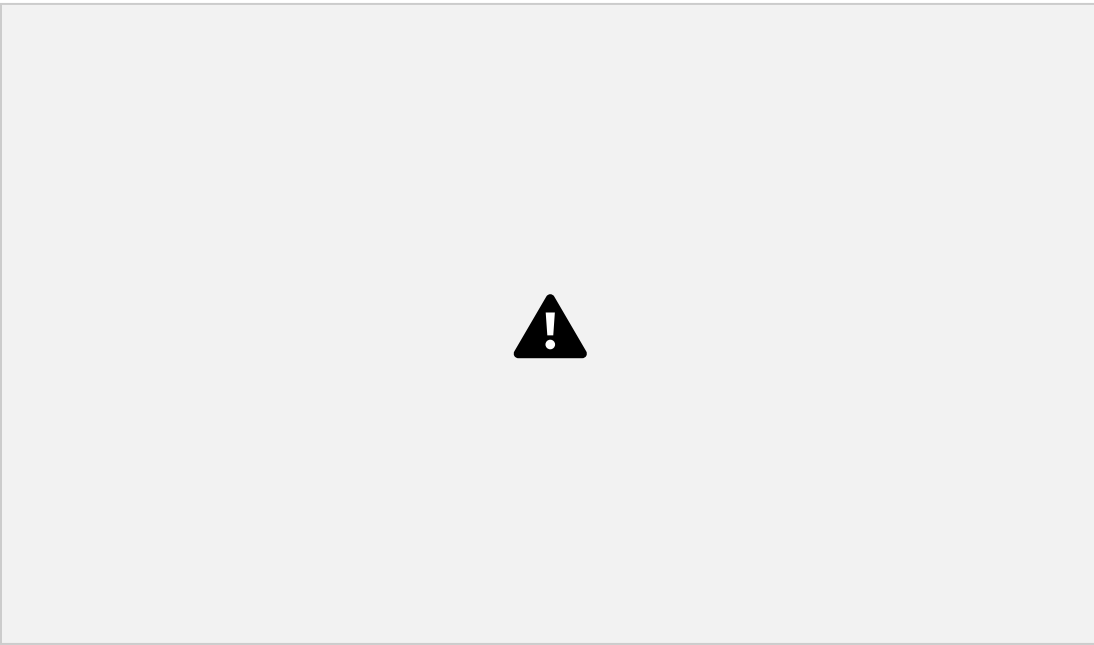
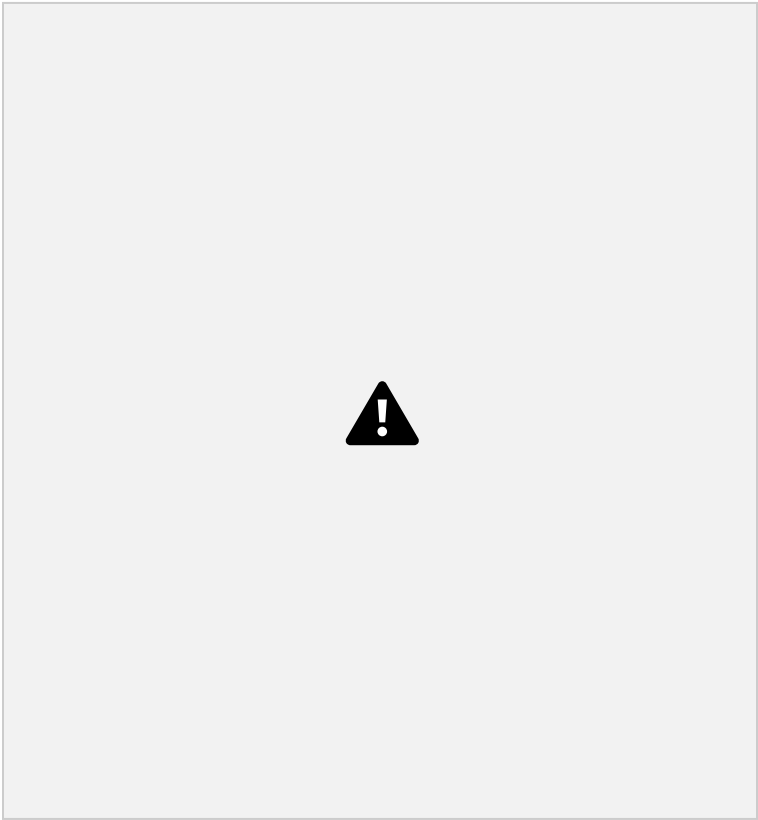


Especifica a coluna de junção para
um relacionamento



Define um relacionamento muitos para-muitos entre duas entidades

Especifica a tabela de junção para um relacionamento muitos-para-muitos





Sem uso do Spring Data JPA



Com uso do Spring Data JPA



O Spring Data JPA fornece uma implementação padrão para as interfaces que estendem JpaRepository. Ou seja, ao definir uma interface de repositório estendendo essa classe, você terá acesso a métodos como save, findAll, findById e delete, por exemplo, prontos para uso, sem a necessidade de implementação manual.

Com uso do Spring Data JPA



Bônus

Inicialização e configuração de projeto

Como criar um projeto?



[HTTPS://START.SPRING.IO/](https://start.spring.io/)

Verificando a versão do JDK



Instalando Gradle e Lombok



